

RentWheelz 2.0

(Vehicle Renting Platform)

Project Report Submitted
in partial fulfillment of the requirements for the award of the

Bachelor of Technology

by

Sritam Sriyanshu Nanda

B522057

Ipsita Hota

B122055

Under the supervision of
Dr. Srichandan Sobhanayak



Department of Computer Science & Engineering
International Institute of Information & Technology

March 2026

APPROVAL OF THE VIVA-VOCE BOARD

March 2026

Certified that the report entitled "RentWheelz 2.0 (Vehicle Renting Platform)" submitted by Ipsita Hota (B122055) and Sritam Sriyanshu Nanda (B522057) to IIIT Bhubaneswar in partial fulfillment of the requirements for the award of the Bachelor of Technology in CSE has been accepted by the examiners during the viva-voce examination held today.

(Supervisor)

(Panel Head)

(Internal Examiner 1)

(Internal Examiner 2)

CERTIFICATE

This is to certify that the report entitled "RentWheelz 2.0 (Vehicle Renting Platform)" submitted Ipsita Hota (B122055) and Sritam Sriyanshu Nanda (B522057) to IIIT Bhubaneswar is a record of Bonafide project work under my supervision, and the report is submitted for end-semester evaluation of the BTech Degree project work.

(Supervisor)

DECLARATION

We certify that

1. The work contained in the report has been done by our group under the general supervision of my supervisor.
2. The work has not been submitted to any other Institute for any degree or diploma.
3. We have followed the guidelines provided by the Institute in writing the thesis.
4. We have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
5. Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them by citing them in the text of the thesis and giving their details in the references.
6. Whenever we have quoted written materials from other sources, we have put them under quotation marks and given due credit to the sources by citing them and giving required details in the references.

Ipsita Hota (B122055)

Sritam Sriyanshu Nanda (B522057)

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to everyone who has contributed to the completion of this project. Our project would not have been possible without the continuous supervision, invaluable guidance, and unwavering support of [Supervisor Name], our esteemed guide throughout this project. We extend our heartfelt thanks to the faculty members of the [Department Name] for their mentorship and support.

Ipsita Hota (B122055)

Sritam Sriyanshu Nanda (B522057)

ABSTRACT

The Online Vehicle Rental and Ride Booking System is designed to connect users with nearby vehicle providers through a real-time digital platform. The system enables passengers to request rides, match with nearby rental drivers, and complete ride transactions efficiently.

The application is developed using a modern full-stack architecture consisting of a React-based frontend and a Node.js backend powered by Express.js and MongoDB. Real-time communication is implemented using Socket.IO to notify drivers about ride requests and update ride status dynamically.

Geolocation services powered by Google Maps APIs allow the system to determine user locations, calculate route distances, and match users with nearby drivers. Security mechanisms such as password hashing and JWT authentication ensure secure communication between clients and servers.

The platform demonstrates how modern web technologies can be combined to build scalable transportation services capable of handling ride requests, driver assignments, and transaction management efficiently.

TABLE OF CONTENTS

1 INTRODUCTION	9
1.1 Problem Statement	9
1.2 Solution	9
2 KEY FEATURES	10
3 TECH STACK	11
3.1 Backend	11
3.2 Frontend	11
3.3 Deployment	11
4 SYSTEM ARCHITECTURE	12
4.1 High-level Architecture	12
5 ADVANTAGES OF THE SYSTEM	13
6 SYSTEM WORKFLOW	13
7 SYSTEM MODULES	14
7.1 User Module	14
7.2 Rental Module	14
7.3 Ride Module	14
7.4 Map Module	14
7.5 Data Flow Overview	15
8 DATA AND DATABASE DESIGN	16
8.1 Database	16
8.2 Core Collections / Models	16
9 MICROSERVICE INTEGRATION	18
9.1 Google Maps Platform	18
9.2 Real-time Event Service (Socket.IO)	18

10	BACKEND API OVERVIEW	19
10.1	User Routes	19
10.2	Rental Routes	19
10.3	Map Routes	19
10.4	Ride Routes	20
11	USER INTERFACE (FRONTEND)	21
11.1	Frontend Structure	21
11.2	Main Pages	21
11.3	API and Socket Integration	21
12	ENVIRONMENT VARIABLES	22
13	HOW EVERYTHING WORKS (CORE LOGIC)	23
13.1	Authentication	23
13.2	Location & Nearby Rental Discovery	23
13.3	Ride Lifecycle	23
13.4	Real-time Chat	23
14	FUTURE WORK	24
15	CONCLUSION	25

1. Introduction

1.1 Problem Statement

Urban mobility and short-distance travel have become increasingly common in modern cities. People frequently need quick and convenient transportation for daily activities such as commuting to work, running errands, attending meetings, or traveling short distances within the city.

Despite the growing demand for such services, traditional vehicle rental workflows still suffer from several significant limitations that reduce their efficiency and user convenience. One of the major issues is that the rental process is often **slow and time-consuming**. In many cases, users must manually check vehicle availability by contacting the renter, visiting a physical office, or browsing through outdated listings. This manual process increases the time required to secure a vehicle and discourages users who need immediate transportation.

- Slow — manual availability checks make the process time-consuming for users.
- Not location-aware — users cannot quickly find nearby vehicle options.
- Poorly coordinated — communication between renter and user is limited, leading to inefficiencies.

1.2 Solution

RentWheelz 2.0 is a real-time vehicle renting platform that connects:

- **Users** — people who want to rent a vehicle for a trip.
- **Rentals / Renters (Captains)** — vehicle owners or providers.

The system provides:

- Authentication for both user roles.
- Location-aware discovery of nearby rentals using MongoDB geospatial queries.
- Real-time ride (rental) lifecycle updates via Socket.IO.
- Google Maps integration for suggestions, coordinates, and routing distance.

2. Key Features

- User and Rental (Captain) authentication system.
- Location-based vehicle discovery using MongoDB geospatial queries.
- Real-time ride request notifications using Socket.IO.
- Ride lifecycle management: request → accept → start → end.
- Google Maps integration for geolocation and route calculation.
- Secure authentication using JWT tokens.
- Password encryption using bcrypt.
- Ride history tracking for both users and rentals.
- Real-time chat between users and rentals.
- Dynamic fare calculation based on ride duration.

The system provides a complete platform for efficient short-distance vehicle rentals. It includes secure authentication for users and rentals using JWT tokens and bcrypt password encryption. Vehicles can be discovered based on location through MongoDB geospatial queries and integrated Google Maps services.

Real-time ride requests, notifications, and chat are enabled using Socket.IO. The platform manages the full ride lifecycle from request to completion, calculates fares dynamically based on ride duration, and maintains detailed ride history for both users and rentals.

3. Tech Stack

3.1 Backend

- **Node.js + Express** — REST APIs for all backend operations.
- **MongoDB + Mongoose** — database and Object Document Mapper (ODM).
- **JWT** — token-based authentication.
- **bcrypt** — password hashing and security.
- **Socket.IO** — real-time events including ride notifications, chat, and location updates.

Google Maps APIs used:

- Geocoding API — address to latitude and longitude conversion.
- Routes API — distance calculation between origin and destination.
- Places Autocomplete — location suggestions for users.

3.2 Frontend

- **React (Vite)** — fast, modern frontend framework.
- **React Router** — client-side routing.
- **Axios** — HTTP API calls to the backend.
- **TailwindCSS** — utility-first styling framework.
- **GSAP** — animations and transitions.
- **Socket.IO Client** — real-time updates and chat.

3.3 Deployment

Vercel configuration exists in the repository for the deployment of the front-end part.
Render configuration exists in the repo for the deployment of the back-end part.

Why Render for Backend but not Vercel?

That's because Vercel doesn't support any real-time connections like Sockets , WebRTC etc. Render has full capability to support any connections whether it's Peer-to-Peer like WebRTC or Sockets like Socket.IO , native WebSockets.

4. System Architecture

4.1 High-level Architecture

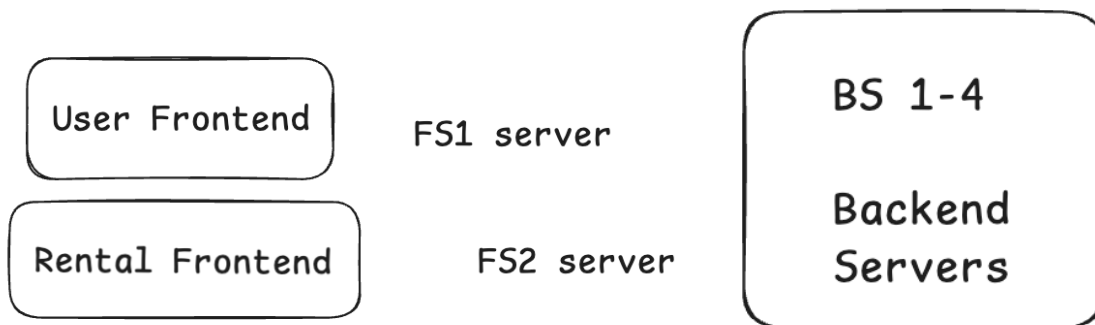
RentWheelz 2.0 is structured as a 2-tier web application with real-time communication.

Frontend (React):

- User UI for ride requests, history, and payment.
- Rental/Captain UI for accepting rides and managing status.
- Uses REST for standard CRUD operations.
- Uses Socket.IO for real-time events such as ride notifications and chat.

Backend (Express):

- REST APIs for authentication, profile management, and ride lifecycle operations.
- MongoDB for persistent data storage.
- Socket.IO server handles:
 - "new ride" broadcast to nearby available rentals.
 - Ride confirmed, started, and ended events.
 - Chat relay between user and rental.
 - Live location updates for real-time tracking.



5. Advantages of the System

- Reduces manual vehicle booking processes, saving time for both users and rental providers.
- Enables real-time driver matching based on proximity and vehicle type.
- Improves transportation accessibility for urban and short-distance travelers.
- Supports a scalable web architecture that can accommodate growing user bases.
- Provides secure authentication and end-to-end communication between parties.
- Efficient ride tracking and history management for both users and rentals.

6. System Workflow

The complete ride lifecycle follows these sequential steps:

- User requests a ride through the platform.
- Backend creates a ride record in the database.
- Backend finds nearby rental providers using a geospatial query.
- Socket.IO sends the ride request notification to matched rental providers.
- A rental provider accepts the ride request.
- The ride starts after OTP verification by the rental provider.
- The ride ends and the fare is automatically calculated based on ride duration.

7. System Modules

The platform is divided into several logical modules that handle different functionalities.

7.1 User Module

Handles passenger operations including:

- Registration and secure login.
- Profile management and updates.
- Ride requests with destination and vehicle type selection.
- Ride history tracking for past trips.

7.2 Rental Module

Manages vehicle providers including:

- Driver registration with vehicle information.
- Vehicle details management (name, color, plate, capacity, type).
- Ride acceptance and real-time status updates.
- Earnings tracking over completed rides.

7.3 Ride Module

Controls the complete ride lifecycle:

- Ride creation with auto-generated OTP.
- Driver assignment based on proximity and vehicle type.
- Ride start validation using OTP.
- Automatic fare calculation on ride end.

7.4 Map Module

Integrates Google Maps services to:

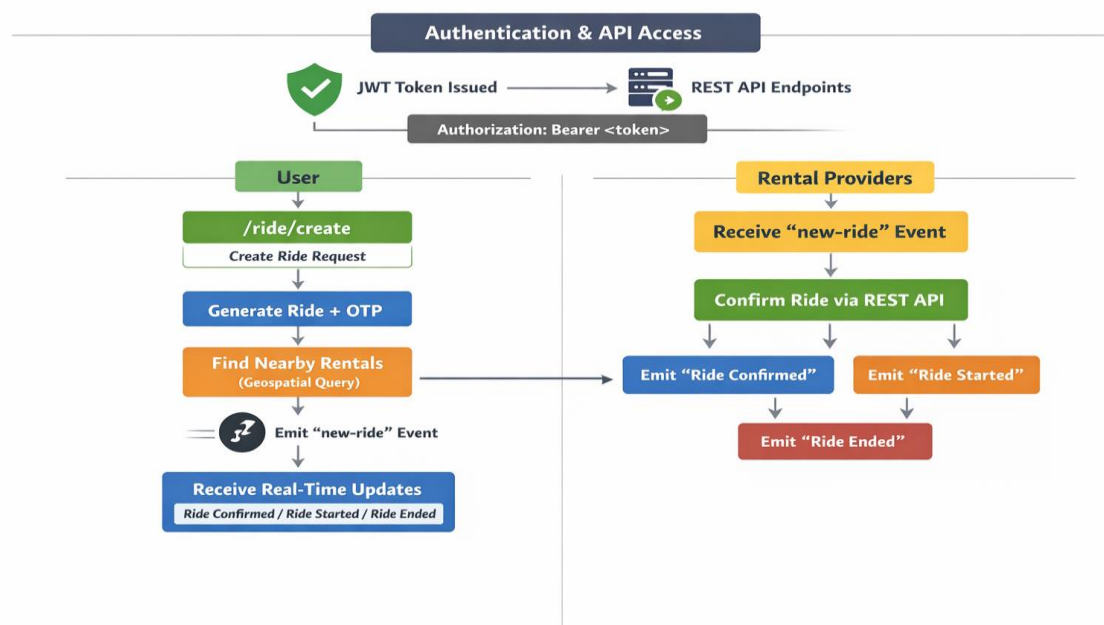
- Convert addresses to geographic coordinates (geocoding).
- Calculate travel distance and estimated time.
- Provide location autocomplete suggestions to users.

7.5 Data Flow Overview

- **Authentication:** REST endpoints (/user/*, /rental/*) issue JWT tokens for secure access.
- **Profile Access:** Frontend sends Authorization: Bearer <token> header with each protected request.
- **Ride Creation:** User calls /ride/create; backend creates a ride record with an OTP, finds nearby rentals using a geospatial query, and emits a Socket.IO 'new-ride' event to matching rental providers.
- **Ride Confirm / Start / End:** Rental confirms via REST; backend emits Socket.IO state change events back to the user in real time.

The system uses **REST APIs and JWT authentication** to secure user and rental access. When a user logs in, the backend issues a **JWT token**, which the frontend sends with each protected request using the **Authorization: Bearer <token>** header.

When a user creates a ride through **/ride/create**, the backend generates a ride with an **OTP**, finds nearby rentals using **geospatial queries**, and sends a **Socket.IO new-ride event**. Rentals confirm rides via REST, and the backend pushes **real-time ride status updates** (confirm, start, end) to the user using **Socket.IO events**.



8. Data and Database Design

8.1 Database

- Database: MongoDB
- Connection is configured via the environment variable DB_CONNECT.

8.2 Core Collections / Models

8.2.1 user

Represents a passenger/renter. Key fields:

- firstName, lastName, email, number
- password — stored as a hashed value (select: false for security).
- socketID — for real-time messaging via Socket.IO.
- location — GeoJSON Point, indexed as 2dsphere for geospatial queries.
- borrowed — count of rides/borrows completed.
- expenses — total amount spent on rides.

8.2.2 rental

Represents a vehicle provider. Key fields:

- firstName, lastName, email, number
- password — stored as a hashed value (select: false).
- socketID — for real-time communication.
- status — 'active' or 'inactive' depending on current availability.
- location — GeoJSON Point, indexed 2dsphere for proximity queries.
- vechile (nested) — name, color, plate, capacity, type (car / bike / scooty).
- earned — total earnings accumulated.
- rented — count of completed rental sessions.

8.2.3 ride

Represents a rental session or ride request. Key fields:

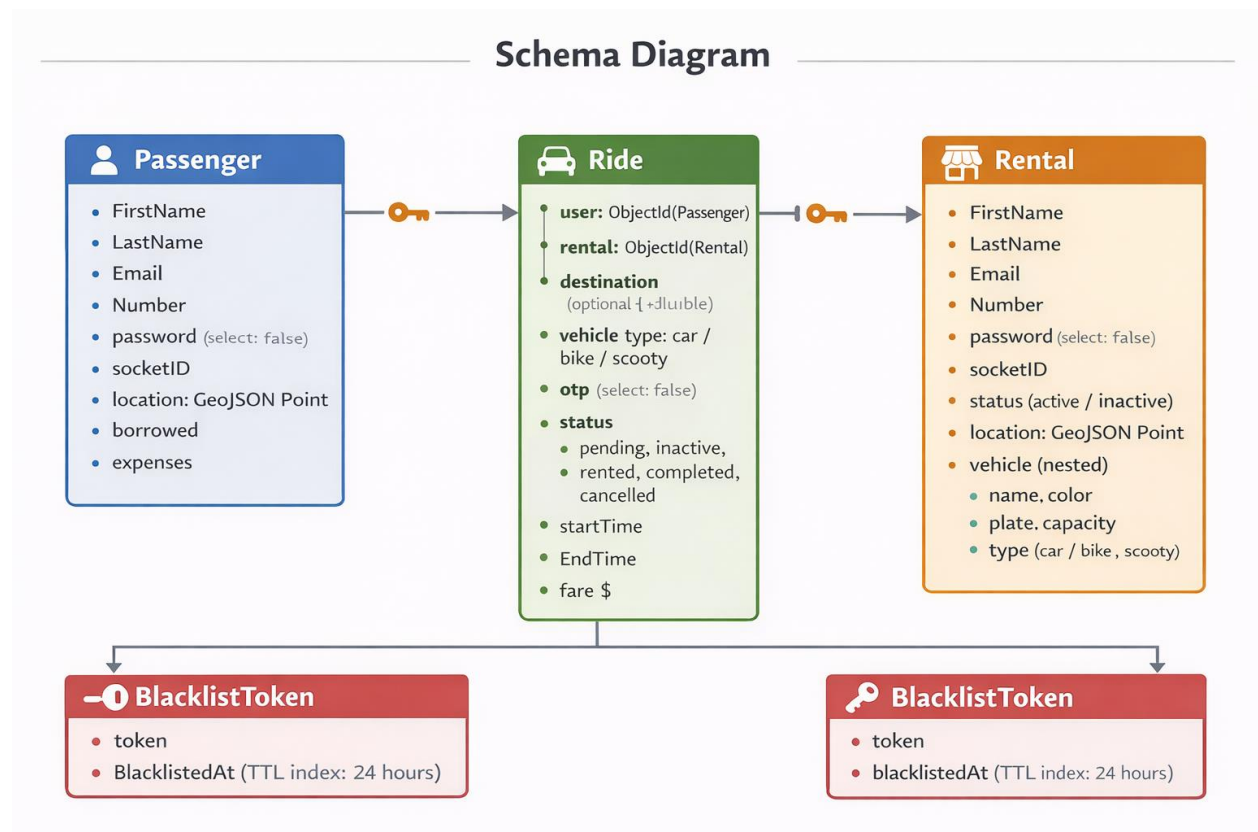
- user — reference to the user collection.
- rental — reference to the rental collection (optional until accepted).
- destination — the target location for the ride.
- vechile — type requested: car, bike, or scooty.

- otp — generated on ride creation (select: false for security).
- status — one of: pending, accepted, rented, completed, cancelled.
- startTime, endTime — timestamps for ride duration tracking.
- fare — computed at ride completion.

8.2.4 blacklistToken

Used to invalidate JWT tokens on user logout.

- token — unique blacklisted token string.
- blacklistedAt — TTL index: token expires automatically after 86400 seconds (24 hours).



9. Microservice Integration

The repository currently runs as a single backend service, but integrates with external services that function as microservice dependencies.

9.1 Google Maps Platform

Used for the following backend operations:

- GET /map/get-coordinates — geocoding of user-entered addresses.
- POST /map/get-distance-time — routing distance calculation for fare estimation.
- GET /map/get-suggestions — places autocomplete for destination input.
- **Environment variable required:** GOOGLE_MAPS_API_KEY

9.2 Real-time Event Service (Socket.IO)

Socket.IO runs inside the backend Node.js process. On connection, clients emit a 'join' event with { userId, userType } to register their socket. The backend stores the socketID in the database for that user or rental provider.

Supported real-time events:

- update-location-user — user sends updated GPS coordinates.
- new-ride — server emits to nearby available rental providers.
- ride-confirmed — server emits to user after rental confirms.
- ride-started — server emits to user when the ride begins.
- ride-ended — server emits to both user and rental on ride completion.
- chat — messages relayed between user and rental in real time.

10. Backend API Overview

Base routes are mounted in Backend/app.js. The four primary route groups are /user, /rental, /map, and /ride.

10.1 User Routes (/user)

POST	/user/register – Register a new user account.
POST	/user/login – Authenticate user and receive JWT.
GET	/user/profile – Retrieve user profile (protected).
GET	/user/logout – Logout and blacklist current token (protected).
PATCH	/user/update – Update user profile details (protected).
GET	/user/history – Get ride history for the user.

10.2 Rental Routes (/rental)

POST	/rental/register – Register a new rental/captain account.
POST	/rental/login – Authenticate rental and receive JWT.
GET	/rental/profile – Retrieve rental profile (protected).
GET	/rental/logout – Logout and blacklist current token.
PATCH	/rental/update – Update rental profile details (protected).
GET	/rental/history – Get ride history for the rental provider.

10.3 Map Routes (/map) — All Protected

GET	/map/get-coordinates?address=... – Geocode an address to lat/lng.
POST	/map/get-distance-time?origin=...&destination=... – Calculate route distance and time.
GET	/map/get-suggestions?input=... – Get place autocomplete suggestions.

10.4 Ride Routes (/ride)

POST	/ride/create – Create a new ride request (protected user).
GET	/ride/get-fare – Get estimated fare for a given route.
POST	/ride/confirm – Confirm a ride (protected rental).
GET	/ride/start-ride?rideId=...&otp=... – Start ride with OTP validation (protected rental).
GET	/ride/end-ride?rideId=... – End the ride and calculate fare (protected rental).

11. User Interface (Frontend)

11.1 Frontend Structure

src/main.jsx wraps the entire application with the following context providers and router:

- **UserContext** — global state for logged-in user.
- **CaptainContext** — global state for logged-in rental/captain.
- **SocketProvider** — initializes and provides the Socket.IO connection.
- **BrowserRouter** — client-side routing via React Router.

11.2 Main Pages

- **Start:** Landing page for the application.
- **User Pages:** UserLogin, UserSignup, Home, profile pages, ride history.
- **Rental/Captain Pages:** CaptainLogin, CaptainSignup, CaptainHome, profile pages, ride history.
- **Ride Lifecycle (User):** Destination search panel → vehicle selection → ride confirmation → waiting state.
- **Ride Lifecycle (Rental):** New ride popup → confirm popup → active riding state.
- **Payment:** Payment page for user and RentalPayment page for rental/captain.

11.3 API and Socket Integration

REST API calls are made using Axios with the following configuration:

- Base URL is set via environment variable: `import.meta.env.VITE_BASE_URL`
- All protected requests include: `Authorization: Bearer <token>`

The Socket.IO connection is initialized in `SocketContext.jsx`:

```
io(import.meta.env.VITE_BASE_URL)
```

12. Environment Variables

12.1 Backend (Backend/.env)

Variable	Description
PORT	Optional: Port number for the backend server.
DB_CONNECT	MongoDB connection string.
JWT_SECRET	Secret key for signing JWT tokens.
GOOGLE_MAPS_API_KEY	API key for all Google Maps Platform services.

12.2 Frontend (frontend/.env)

Variable	Description
VITE_BASE_URL	Backend base URL for all API and socket connections.
VITE_GOOGLE_MAPS_API_KEY	Google Maps API key if used directly in UI components.

13. How Everything Works (Core Logic)

13.1 Authentication

- Registration stores a bcrypt-hashed password for security.
- Login validates the password and issues a JWT token with a 24-hour expiry.
- Protected routes use authentication middleware: authUser (for users) and authRental (for rental providers).
- Logout blacklists the current token using the blacklistToken collection with a TTL of 24 hours.

13.2 Location and Nearby Rental Discovery

- User location is retrieved via browser geolocation API and emitted through Socket.IO for real-time tracking.
- Rental documents store GeoJSON coordinates and are indexed with 2dsphere for efficient spatial queries.
- On ride creation, the backend searches for available rental providers within a defined radius using \$geoWithin / \$centerSphere MongoDB operators.

13.3 Ride Lifecycle

- **Create:** The user creates a ride record with status 'pending' and an auto-generated OTP is assigned.
- **Notify Rentals:** The backend emits a 'new-ride' Socket.IO event to nearby rental providers that match the requested vehicle type.
- **Confirm:** The rental confirms the ride; status becomes 'accepted' and the rental's status changes to 'inactive' to prevent double-booking.
- **Start:** The rental starts the ride by validating the OTP provided by the user.
- **End:** Fare is computed using time-based logic combined with a base vehicle price; all statuses, earnings, and expense totals are updated accordingly.

13.4 Real-time Chat

- Messages are sent via the 'chat' Socket.IO event from either user or rental.
- The server looks up the target socketId stored in the database and relays the message directly to the recipient's socket connection.

14. Future Work

- **Payments Gateway Integration:** Integrate Razorpay or Stripe for real-time payment processing instead of a post-ride summary only approach.
- **Better Ride State Machine:** Improve separation between 'accepted' and 'rented' states, and introduce cancellation rules and timeout mechanisms.
- **Admin Dashboard:** Build a dedicated admin panel for driver verification, vehicle approvals, and platform-wide analytics.
- **Stronger Security:** Implement refresh tokens, CSRF hardening for cookie-based auth, and request rate limiting.
- **Testing:** Add unit tests for service layers and integration tests for all API routes to ensure reliability.
- **Production-ready Configuration:** Enforce CORS allowlists, add structured logging, and implement environment variable validation for safer deployments.

The platform can be further improved by integrating several advanced features and best practices. **Payment gateway integration** using services like **Razorpay or Stripe** would enable secure and real-time payment processing instead of relying only on post-ride summaries. The **ride state machine** can be enhanced by clearly separating states such as *accepted* and *rented*, while also introducing ride cancellation rules and timeout mechanisms for better workflow management.

Developing an **admin dashboard** would allow administrators to verify drivers, approve vehicles, and monitor platform-wide analytics, improving operational control. Security can be strengthened by implementing **refresh tokens, CSRF protection for cookie-based authentication, and API rate limiting** to prevent abuse.

To ensure reliability and maintainability, the project should include **unit tests for service layers and integration tests for API routes**. Finally, adopting **production-ready configurations** such as CORS allowlists, structured logging, and environment variable validation would help ensure safer and more stable deployments.

15. Conclusion

RentWheelz 2.0 presents a comprehensive **full-stack vehicle rental platform** that demonstrates how modern web technologies can be combined to build a scalable and real-time transportation service. The system integrates **RESTful APIs** for core backend operations such as authentication, ride creation, and ride lifecycle management. **MongoDB with geospatial indexing** enables efficient discovery of nearby rental providers based on the user's location, ensuring quick and accurate matching between users and available vehicles.

To support real-time interaction, the platform utilizes **Socket.IO**, which enables instant ride notifications, ride status updates, and real-time communication between users and rental providers. In addition, **Google Maps APIs** are integrated to provide reliable geolocation services, route calculations, and destination mapping, enhancing the overall user experience and navigation accuracy.

The application is designed with **separate portals for users and rental providers**, ensuring role-based functionality and smooth interaction between both sides of the platform. The system supports the **complete ride lifecycle**, including ride request, confirmation, ride start, ride completion, and automatic fare calculation.

From an architectural perspective, the backend follows a **modular structure with clearly separated routes, controllers, services, and models**, promoting maintainability and clean code practices. Security is strengthened through **JWT-based authentication, password hashing, and token blacklisting mechanisms**, ensuring safe session handling. Overall, RentWheelz 2.0 demonstrates a well-structured, scalable, and production-ready approach to building a modern real-time vehicle rental service.

References

- Socket.IO Documentation. Real-time bidirectional event-based communication. <https://socket.io/docs/>
- MongoDB Documentation. Geospatial Queries. <https://www.mongodb.com/docs/manual/geospatial-queries/>
- Google Maps Platform. Maps JavaScript API and Geocoding API. <https://developers.google.com/maps/documentation>
- JSON Web Tokens (JWT). Introduction to JSON Web Tokens. <https://jwt.io/introduction/>
- React Documentation. React — A JavaScript library for building user interfaces. <https://react.dev/>
- Express.js Documentation. Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com/>
- Mongoose Documentation. Elegant MongoDB object modeling for Node.js. <https://mongoosejs.com/docs/>
- Vercel Documentation. Deploying Node.js applications. <https://vercel.com/docs>